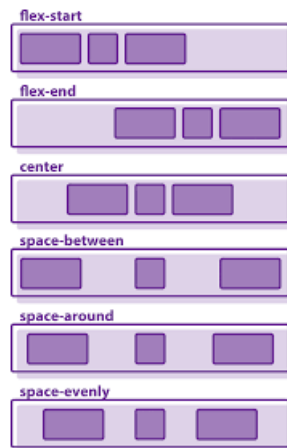


Vinsup Skill Academy CSS

Flex



Before the Flexbox Layout module, there were four layout modes:

- Block, for sections in a webpage
- Inline, for text
- Table, for two-dimensional table data
- Positioned, for explicit position of an element

The Flexible Box Layout Module, makes it easier to design flexible responsive layout structure without using float or positioning.

Flexbox Elements

To start using the Flexbox model, you need to first define a flex container.



The element above represents a flex container (the blue area) with three flex items.

Flex Container

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  margin: 10px;
  padding: 20px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>Create a Flex Container</h1>

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

<p>A Flexible Layout must have a parent element with the <em>display</em>
m> property set to <em>flex</em>.</p>

<p>Direct child elements(s) of the flexible container automatically become
s flexible items.</p>

</body>
</html>
```

Create a Flex Container



A Flexible Layout must have a parent element with the *display* property set to *flex*.

Direct child element(s) of the flexible container automatically becomes flexible items.

The flex container properties are:

- `flex-direction`
- `flex-wrap`
- `flex-flow`
- `justify-content`
- `align-items`
- `align-content`

Flex Direction:

The `flex-direction` property accepts four possible values:

- `row` : same as text direction (default)
- `row-reverse` : opposite to text direction.
- `column` : same as row but top to bottom.
- `column-reverse` : same as row-reverse top to bottom.

Example

The `column` value stacks the flex items vertically (from top to bottom):

```
<!DOCTYPE html>  
<html>
```

```

<head>
<style>
.flex-container {
  display: flex;
  flex-direction: column;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The flex-direction Property</h1>

<p>The "flex-direction: column;" stacks the flex items vertically (from top to bottom):</p>

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

</body>
</html>

```

The flex-direction Property

The "flex-direction: column;" stacks the flex items vertically (from top to bottom):



Example

The `column-reverse` value stacks the flex items vertically (but from bottom to top):

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  flex-direction: column;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
```

```
</style>
</head>
<body>

<h1>The flex-direction Property</h1>

<p>The "flex-direction: column;" stacks the flex items vertically (from top to bottom):</p>

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

</body>
</html>
```

The flex-direction Property

The "flex-direction: column;" stacks the flex items vertically (from top to bottom):



Example

The `row` value stacks the flex items horizontally (from left to right):

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  flex-direction: row;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The flex-direction Property</h1>

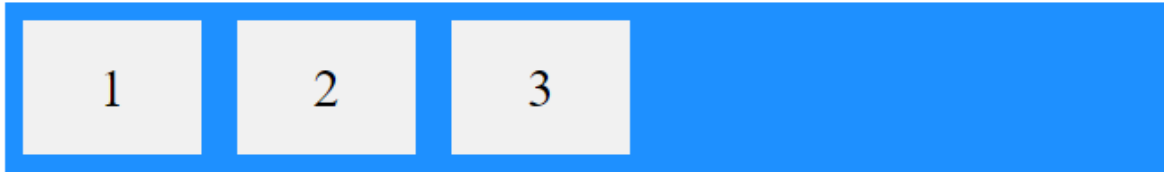
<p>The "flex-direction: row;" stacks the flex items horizontally (from left to
right):</p>

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

</body>
</html>
```

The flex-direction Property

The "flex-direction: row;" stacks the flex items horizontally (from left to right):



Example

The `row-reverse` value stacks the flex items horizontally (but from right to left):

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  flex-direction: row-reverse;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The flex-direction Property</h1>
```

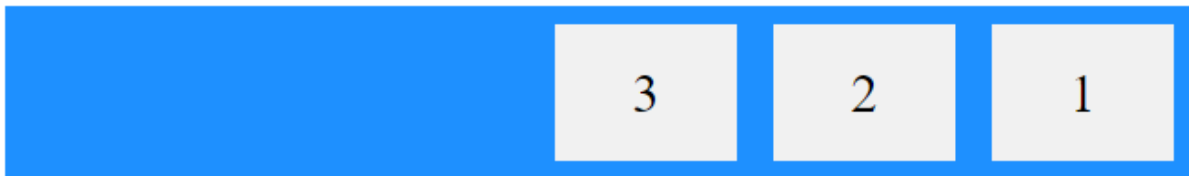

<p>The "flex-direction: row-reverse;" stacks the flex items horizontally (but from right to left):</p>

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

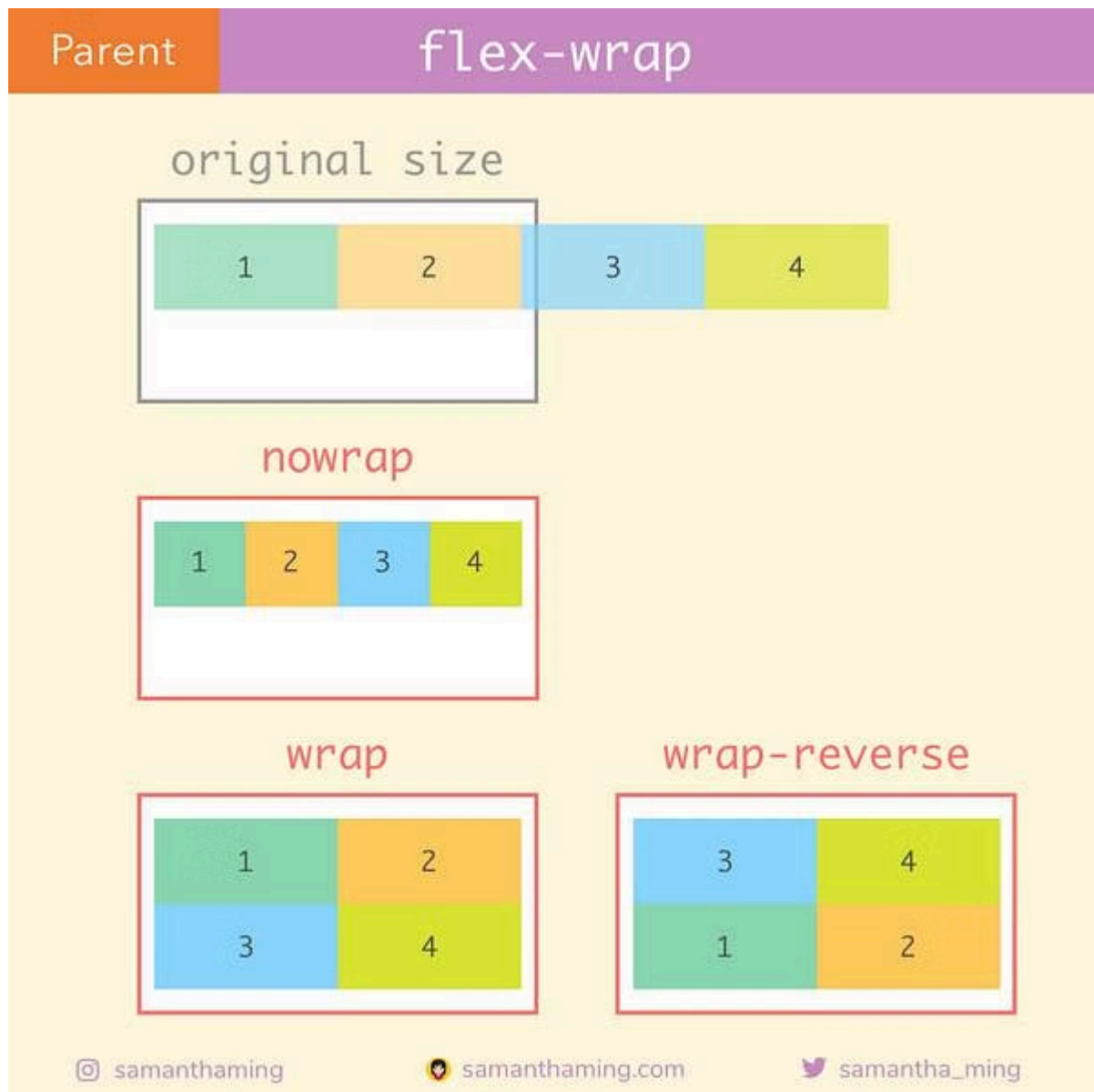
</body>
</html>
```

The flex-direction Property

The "flex-direction: row-reverse;" stacks the flex items horizontally (but from right to left):



flex-wrap Property



- The flex-wrap CSS property **sets whether flex items are forced onto one line or can wrap onto multiple lines.**
- If wrapping is allowed, it sets the direction that lines are stacked.

Example for **wrap** :

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
```

```
flex-wrap: wrap;
background-color: DodgerBlue;
}
```

```
.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>The flex-wrap Property</h1>
```

<p>The "flex-wrap: wrap;" specifies that the flex items will wrap if necessary:</p>

```
<div class="flex-container">
```

```
<div>1</div>
```

```
<div>2</div>
```

```
<div>3</div>
```

```
<div>4</div>
```

```
<div>5</div>
```

```
<div>6</div>
```

```
<div>7</div>
```

```
<div>8</div>
```

```
<div>9</div>
```

```
<div>10</div>
```

```
<div>11</div>
```

```
<div>12</div>
```

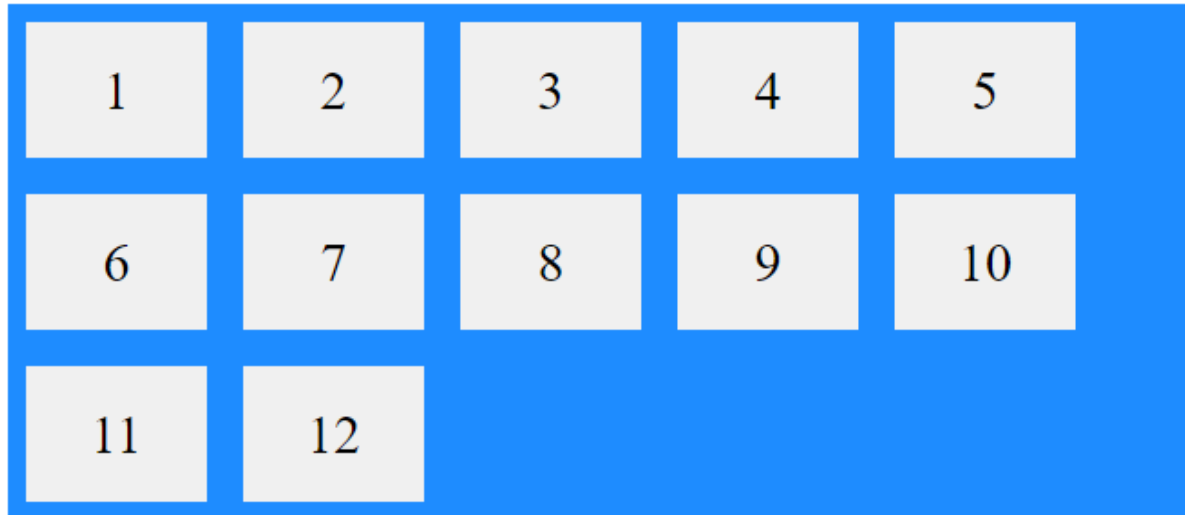
```
</div>
```

<p>Try resizing the browser window.</p>

```
</body>
</html>
```

The flex-wrap Property

The "flex-wrap: wrap;" specifies that the flex items will wrap if necessary:



Try resizing the browser window.

Example for nowrap

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  flex-wrap: nowrap;
  background-color: DodgerBlue;
}

.flex-container>div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
```

```
text-align: center;
line-height: 75px;
font-size: 30px;
}
</style>
</head>
<body>

<h1>The flex-wrap Property</h1>

<p>The "flex-wrap: nowrap;" specifies that the flex items will not wrap (this
is default):</p>

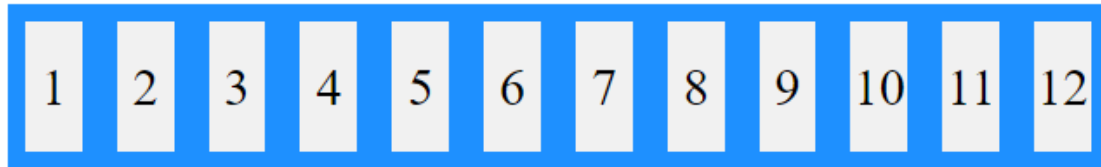
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
  <div>9</div>
  <div>10</div>
  <div>11</div>
  <div>12</div>
</div>

<p>Try resizing the browser window.</p>

</body>
</html>
```

The flex-wrap Property

The "flex-wrap: nowrap;" specifies that the flex items will not wrap (this is default):



Try resizing the browser window.

Example for `wrap-reverse`

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  flex-wrap: wrap-reverse;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The flex-wrap Property</h1>
```

The "flex-wrap: wrap-reverse;" specifies that the flex items will wrap if necessary, in reverse order:

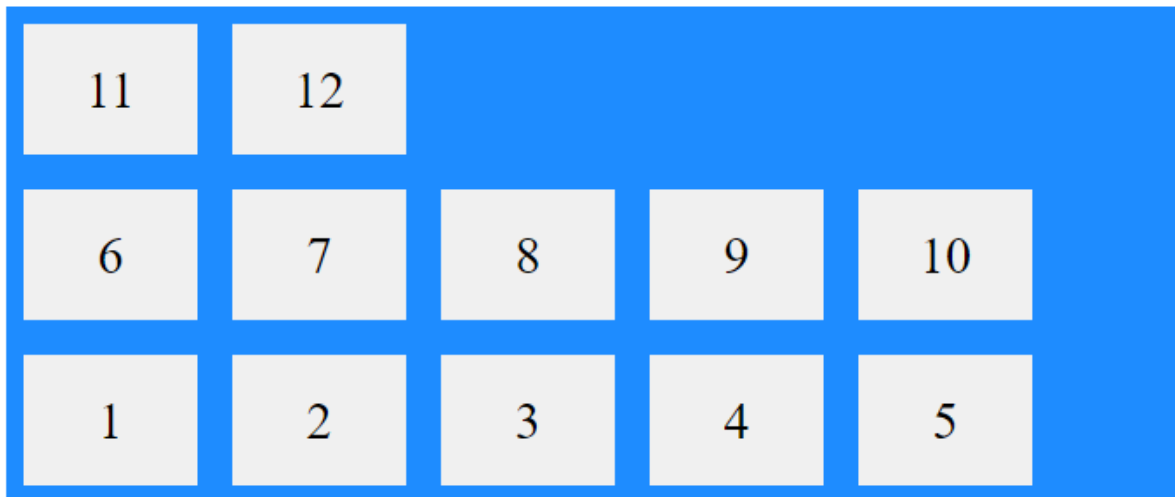
```
<div class="flex-container">  
  <div>1</div>  
  <div>2</div>  
  <div>3</div>  
  <div>4</div>  
  <div>5</div>  
  <div>6</div>  
  <div>7</div>  
  <div>8</div>  
  <div>9</div>  
  <div>10</div>  
  <div>11</div>  
  <div>12</div>  
</div>
```

Try resizing the browser window.

```
</body>  
</html>
```

The flex-wrap Property

The "flex-wrap: wrap-reverse;" specifies that the flex items will wrap if necessary, in reverse order:



Try resizing the browser window.

Flex -Flow

The `flex-flow` property is a shorthand property for:

- flex-direction
- flex-wrap

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
#main {
  width: 200px;
  height: 200px;
  border: 1px solid #c3c3c3;
  display: flex;
  flex-flow: row-reverse wrap;
}

#main div {
```



```

width: 50px;
height: 50px;
}
</style>
</head>
<body>

<h1>The flex-flow Property</h1>

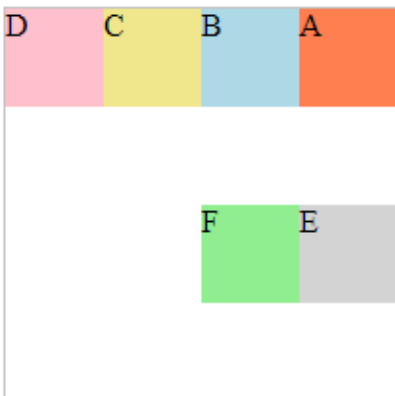
<div id="main">
  <div style="background-color:coral;">A</div>
  <div style="background-color:lightblue;">B</div>
  <div style="background-color:khaki;">C</div>
  <div style="background-color:pink;">D</div>
  <div style="background-color:lightgrey;">E</div>
  <div style="background-color:lightgreen;">F</div>
</div>

<p><b>Note:</b> Internet Explorer 10 and earlier versions do not support the flex-flow property.</p>

</body>
</html>

```

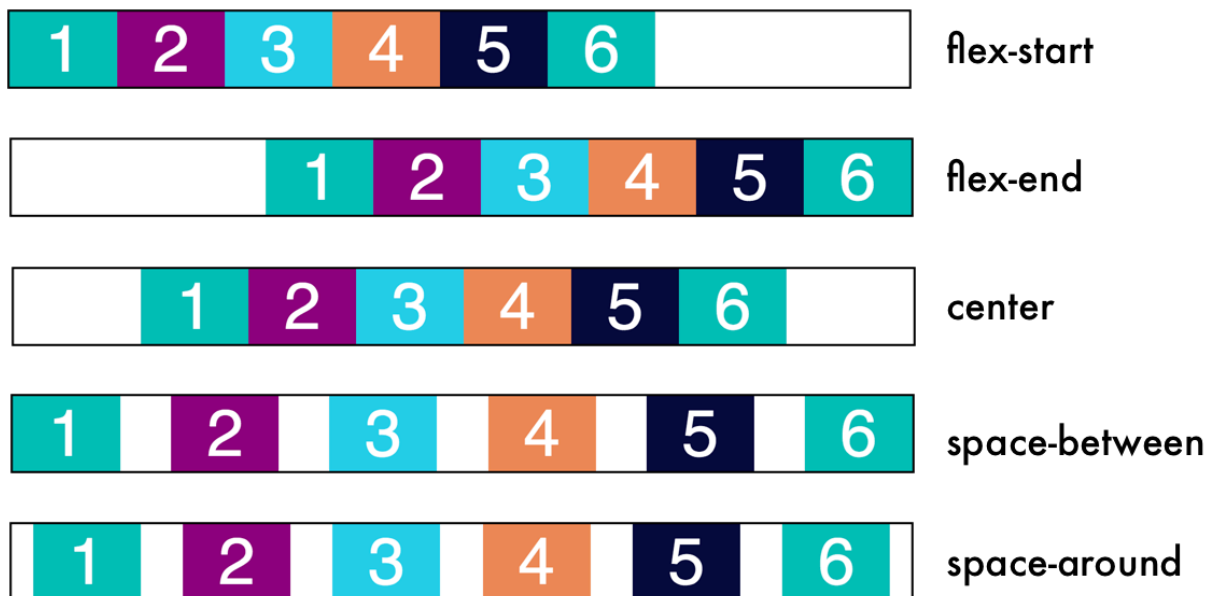
The flex-flow Property



Note: Internet Explorer 10 and earlier versions do not support the flex-flow property.

justify-content Property:

- The `justify-content` property in CSS is used to align the items inside a flex container along the main axis (horizontally in a row layout or vertically in a column layout).
- This property defines how the space is distributed between and around the flex items.



Syntax:

```
justify-content: flex-start | flex-end | center |  
space-between | space-around | space-evenly;
```

Values:

1. **flex-start**: Items are packed toward the start of the flex container (default).
2. **flex-end**: Items are packed toward the end of the flex container.
3. **center**: Items are centered along the main axis.
4. **space-between**: Items are evenly distributed; the first item is at the start, and the last item is at the end.
5. **space-around**: Items are evenly distributed with equal space around them.
6. **space-evenly**: Items are evenly distributed with equal space between them.

Example for center

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  justify-content: center;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The justify-content Property</h1>

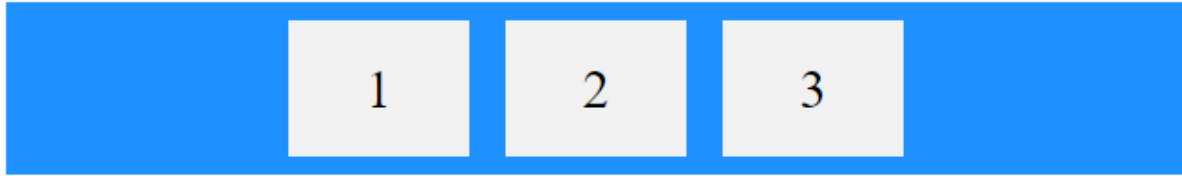
<p>The "justify-content: center;" aligns the flex items at the center of the container:</p>

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

</body>
</html>
```

The justify-content Property

The "justify-content: center;" aligns the flex items at the center of the container:



Example for **flex-start**

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  justify-content: flex-start;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The justify-content Property</h1>

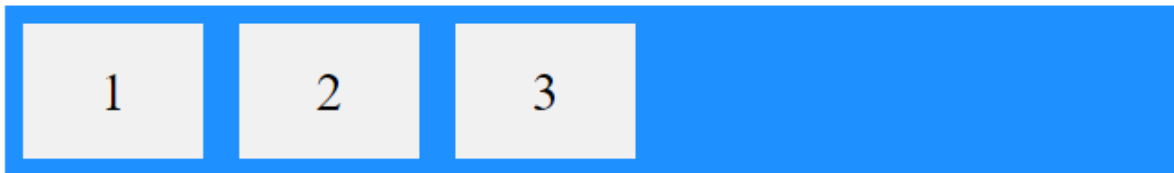
<p>The "justify-content: flex-start;" aligns the flex items at the beginning o
f the container (this is default):</p>
```

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

</body>
</html>
```

The justify-content Property

The "justify-content: flex-start;" aligns the flex items at the beginning of the container (this is default):



Example for `flex-end`

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  justify-content: flex-end;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
```

```

text-align: center;
line-height: 75px;
font-size: 30px;
}
</style>
</head>
<body>

<h1>The justify-content Property</h1>

<p>The "justify-content: flex-end;" aligns the flex items at the end of the container:</p>

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

</body>
</html>

```

The justify-content Property

The "justify-content: flex-end;" aligns the flex items at the end of the container:



Example for space-around

```

<!DOCTYPE html>
<html>
<head>

```

```
<style>
.flex-container {
  display: flex;
  justify-content: space-around;
  background-color: DodgerBlue;
}
```

```
.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>The justify-content Property</h1>
```

<p>The "justify-content: space-around;" displays the flex items with space before, between, and after the lines:</p>

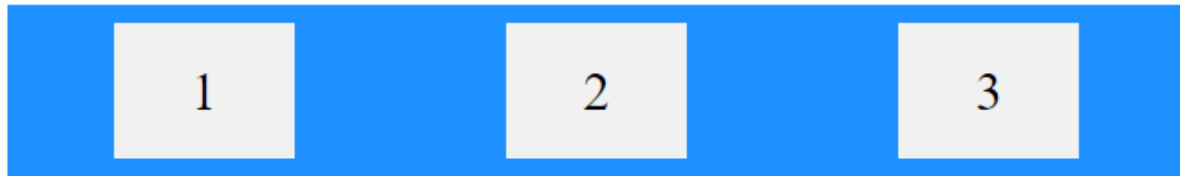
```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>
```

```
</body>
```

```
</html>
```

The justify-content Property

The "justify-content: space-around;" displays the flex items with space before, between, and after the lines:



Example for space-between

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  justify-content: space-between;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The justify-content Property</h1>

<p>The "justify-content: space-between;" displays the flex items with spac
```

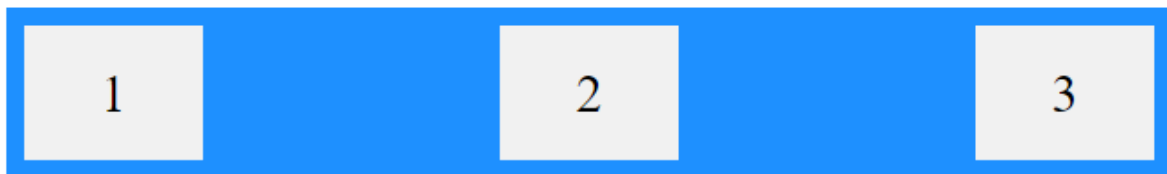

e between the lines:</p>

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>

</body>
</html>
```

The justify-content Property

The "justify-content: space-between;" displays the flex items with space between the lines:



align-items Property:

The `align-items` property in CSS is used to align flex items along the **cross axis** of the flex container (perpendicular to the main axis). If the main axis is horizontal (using `flex-direction: row`), the cross axis will be vertical, and vice versa. It defines how flex items are positioned within the flex container along this axis.

Key Concepts:

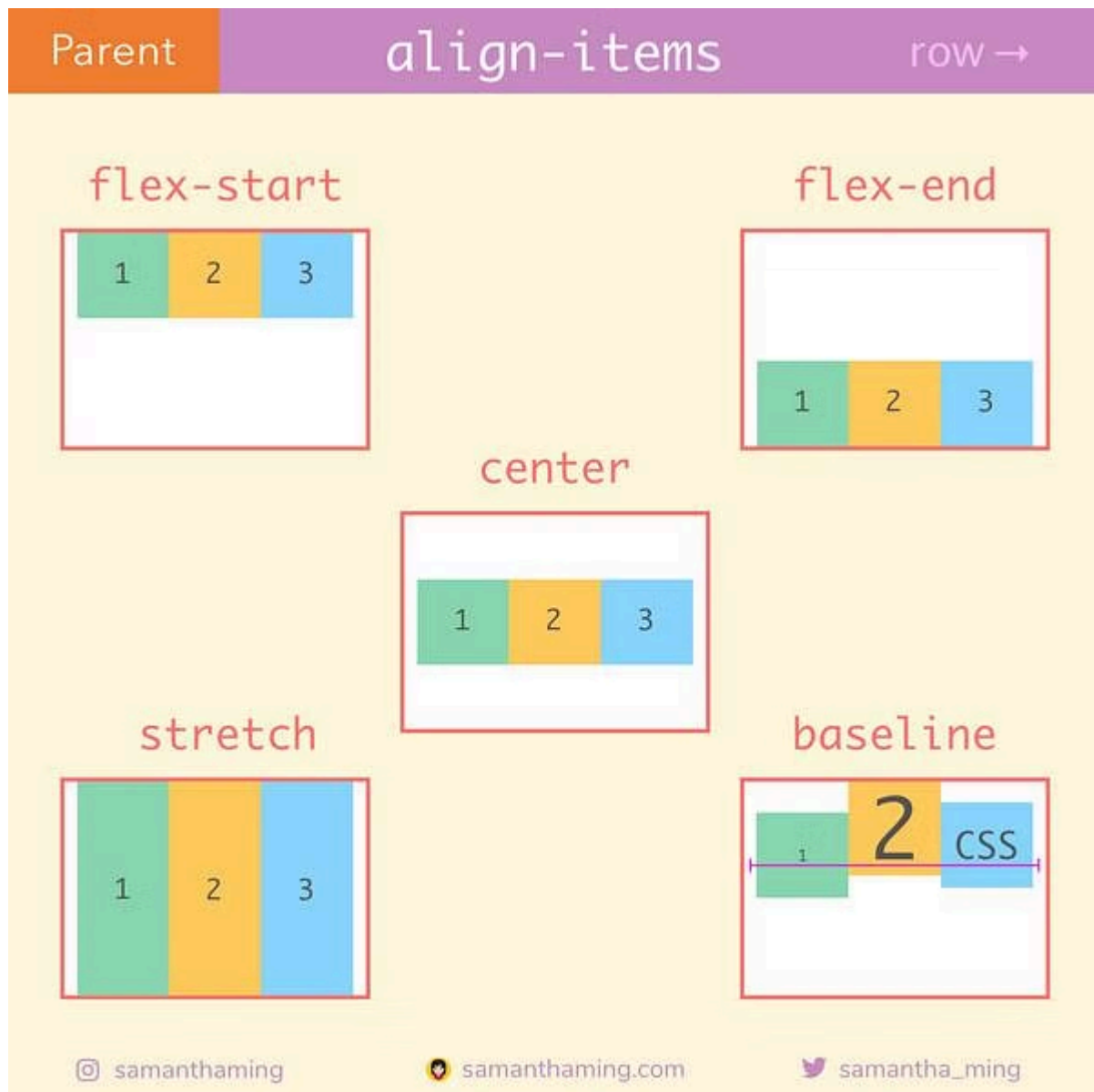
1. Main Axis:

- The main axis is defined by the direction of the flex container, which is controlled by the `flex-direction` property.
- If `flex-direction` is set to `row` (default), the main axis is **horizontal** (left to right).

- If `flex-direction` is set to `column`, the main axis becomes **vertical** (top to bottom).

Possible Values:

1. **stretch** (default): Items stretch to fill the container along the cross axis (height in a row or width in a column).
2. **flex-start**: Items are aligned to the start of the cross axis (top-aligned in a row, left-aligned in a column).
3. **flex-end**: Items are aligned to the end of the cross axis (bottom-aligned in a row, right-aligned in a column).
4. **center**: Items are centered along the cross axis.
5. **baseline**: Items are aligned along their text baselines.



Example for **center**

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  height: 200px;
  align-items: center;
  background-color: DodgerBlue;
}
```

```
.flex-container > div {  
  background-color: #f1f1f1;  
  width: 100px;  
  margin: 10px;  
  text-align: center;  
  line-height: 75px;  
  font-size: 30px;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>The align-items Property</h1>
```

```
<p>The "align-items: center;" aligns the flex items in the middle of the container:</p>
```

```
<div class="flex-container">
```

```
<div>1</div>
```

```
<div>2</div>
```

```
<div>3</div>
```

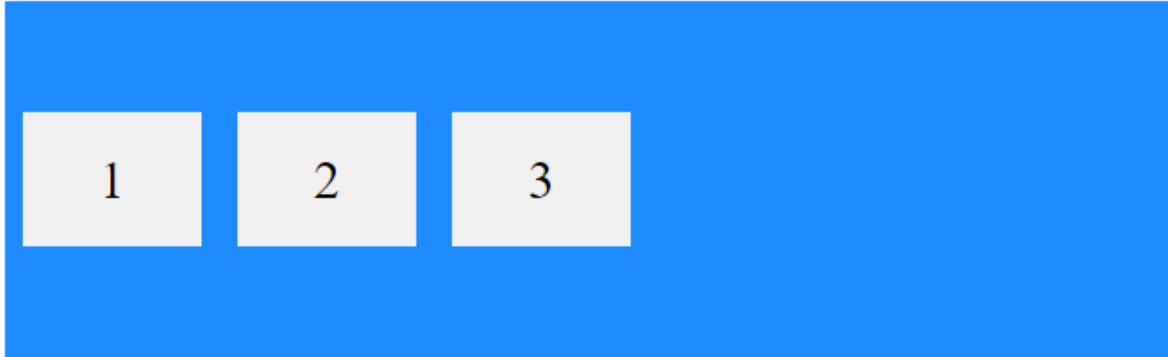
```
</div>
```

```
</body>
```

```
</html>
```

The align-items Property

The "align-items: center;" aligns the flex items in the middle of the container:



Example for **flex-start** :

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  height: 200px;
  align-items: flex-start;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>
```

```
<h1>The align-items Property</h1>
```

```
<p>The "align-items: flex-start;" aligns the flex items at the top of the container:</p>
```

```
<div class="flex-container">
```

```
<div>1</div>
```

```
<div>2</div>
```

```
<div>3</div>
```

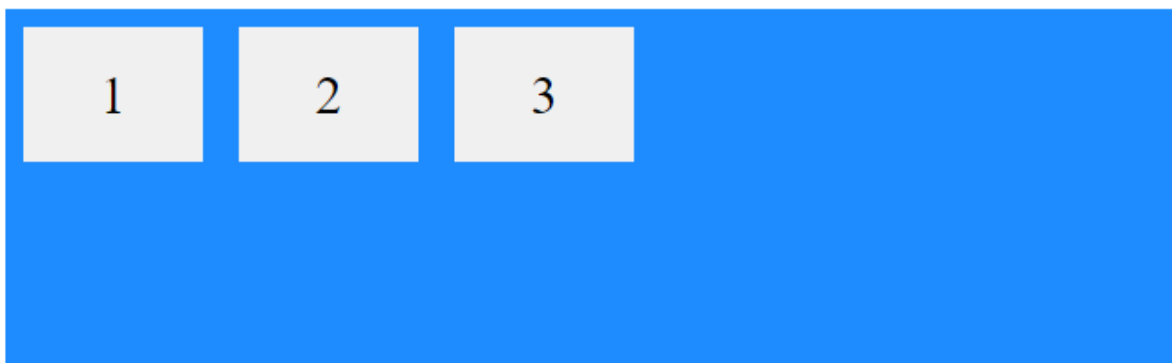
```
</div>
```

```
</body>
```

```
</html>
```

The align-items Property

The "align-items: flex-start;" aligns the flex items at the top of the container:



Example for flex-end

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  height: 200px;
  align-items: flex-end;
```

```
background-color: DodgerBlue;  
}
```

```
.flex-container > div {  
  background-color: #f1f1f1;  
  width: 100px;  
  margin: 10px;  
  text-align: center;  
  line-height: 75px;  
  font-size: 30px;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>The align-items Property</h1>
```

<p>The "align-items: flex-end;" aligns the flex items at the bottom of the container:</p>

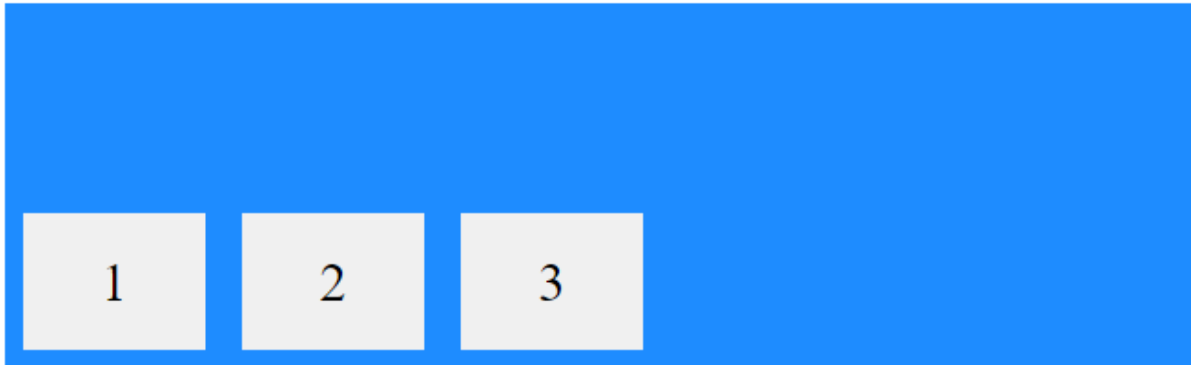
```
<div class="flex-container">  
  <div>1</div>  
  <div>2</div>  
  <div>3</div>  
</div>
```

```
</body>
```

```
</html>
```

The align-items Property

The "align-items: flex-end;" aligns the flex items at the bottom of the container:



Example for **stretch** :

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  height: 200px;
  align-items: stretch;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
  text-align: center;
  line-height: 75px;
  font-size: 30px;
}
</style>
</head>
<body>
```



```
<h1>The align-items Property</h1>
```

```
<p>The "align-items: stretch;" stretches the flex items to fill the container (this is default):</p>
```

```
<div class="flex-container">
```

```
<div>1</div>
```

```
<div>2</div>
```

```
<div>3</div>
```

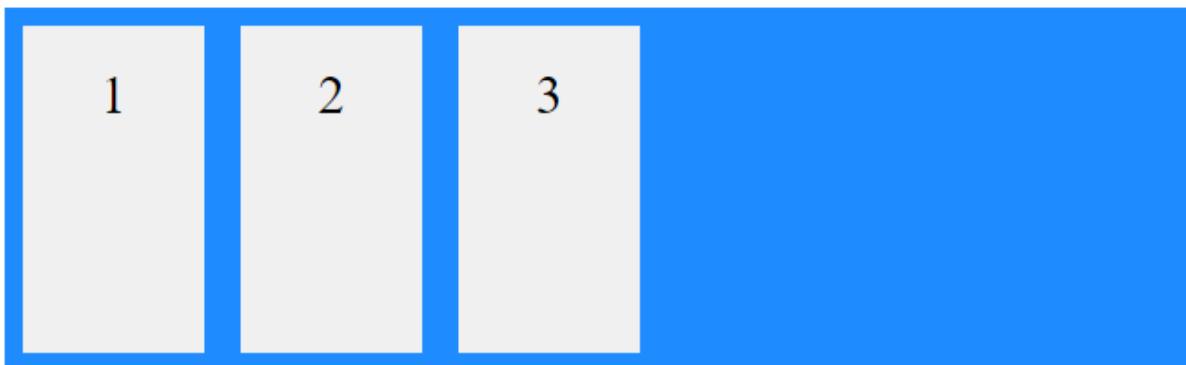
```
</div>
```

```
</body>
```

```
</html>
```

The align-items Property

The "align-items: stretch;" stretches the flex items to fill the container (this is default):



Example for **baseline** :

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
.flex-container {
```

```
display: flex;
```

```
height: 200px;
```

```
align-items: baseline;
```

```

background-color: DodgerBlue;
}

.flex-container > div {
background-color: #f1f1f1;
width: 100px;
margin: 10px;
text-align: center;
line-height: 75px;
font-size: 30px;
}
</style>
</head>
<body>
<h1>The align-items Property</h1>

```

<p>The "align-items: baseline;" aligns the flex items such as their baseline s aligns:</p>

```

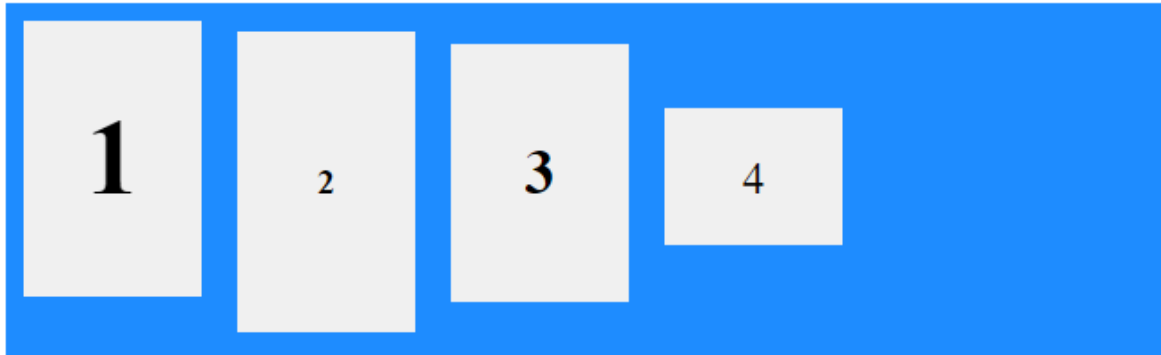
<div class="flex-container">
  <div><h1>1</h1></div>
  <div><h6>2</h6></div>
  <div><h3>3</h3></div>
  <div><small>4</small></div>
</div>

</body>
</html>

```

The align-items Property

The "align-items: baseline;" aligns the flex items such as their baselines aligns:



align-content Property:

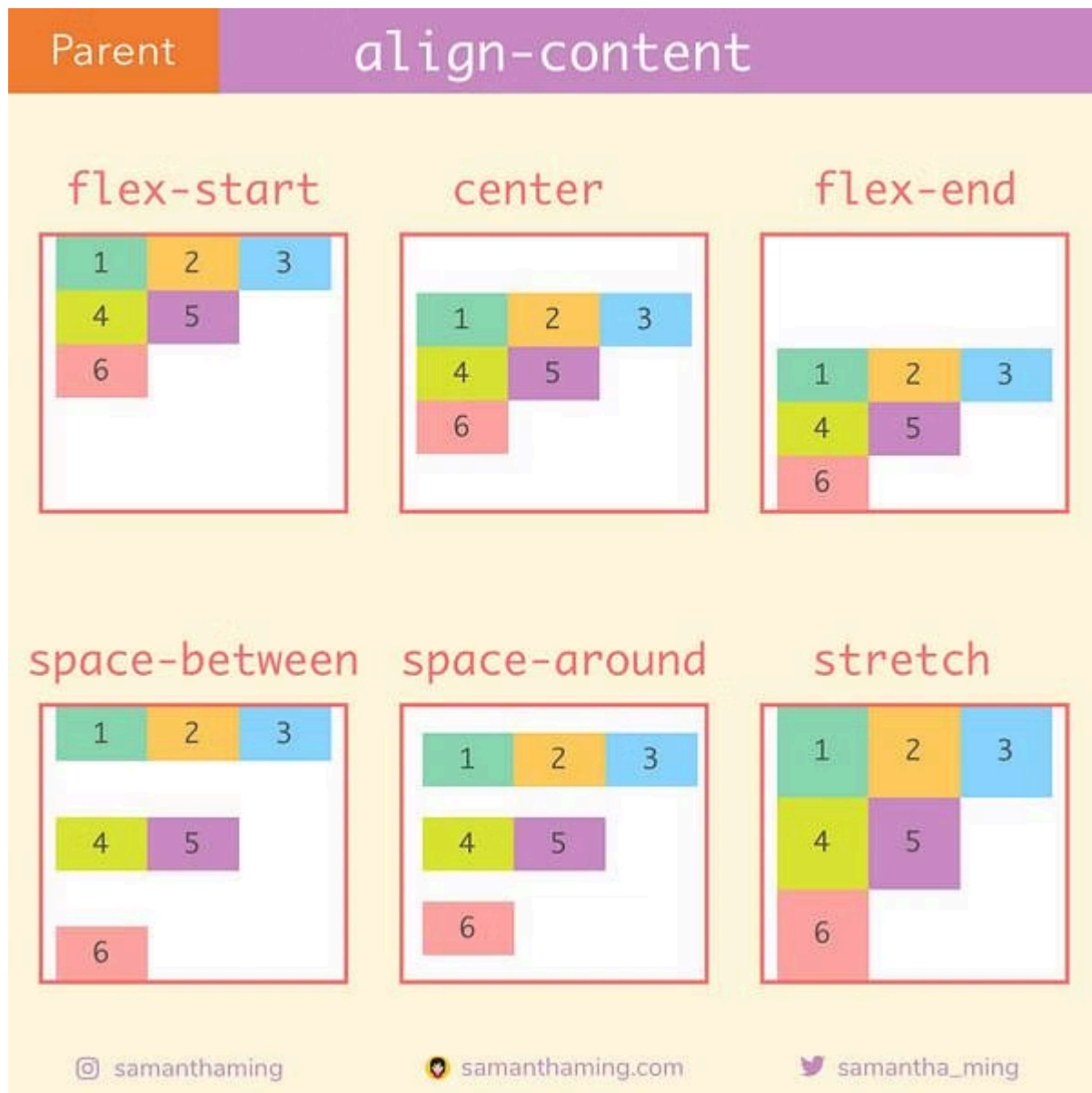
The `align-content` property in CSS is used to align **flex lines** when there is **extra space** in a flex container with multiple lines (i.e., when `flex-wrap` is used). It controls the distribution of space **between and around** flex lines on the **cross axis**.

This property has no effect when there is only one line of flex items. It is mainly useful for **multi-line flex containers**, where flex items wrap onto additional lines (like rows or columns, depending on the flex direction).

Values for `align-content` :

1. **stretch** (default): The flex lines stretch to take up the remaining space.
2. **flex-start**: The flex lines are packed toward the start of the container (top for row, left for column).
3. **flex-end**: The flex lines are packed toward the end of the container (bottom for row, right for column).
4. **center**: The flex lines are packed toward the center of the container.
5. **space-between**: The flex lines are evenly distributed, with no space at the start or end of the container.
6. **space-around**: The flex lines are evenly distributed with equal space around them.

7. **space-evenly**: Flex lines are evenly distributed with equal space between them and around them.



•

Example for **space-between**

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
```

```
display: flex;  
height: 600px;  
flex-wrap: wrap;  
align-content: space-between;  
overflow: scroll;  
background-color: DodgerBlue;  
}
```

```
.flex-container > div {  
background-color: #f1f1f1;  
width: 100px;  
margin: 10px;  
text-align: center;  
line-height: 75px;  
font-size: 30px;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>The align-content Property</h1>
```

<p>The "align-content: space-between;" displays the flex lines with equal space between them:</p>

```
<div class="flex-container">
```

```
<div>1</div>
```

```
<div>2</div>
```

```
<div>3</div>
```

```
<div>4</div>
```

```
<div>5</div>
```

```
<div>6</div>
```

```
<div>7</div>
```

```
<div>8</div>
```

```
<div>9</div>
```

```
<div>10</div>
```

```
<div>11</div>
```

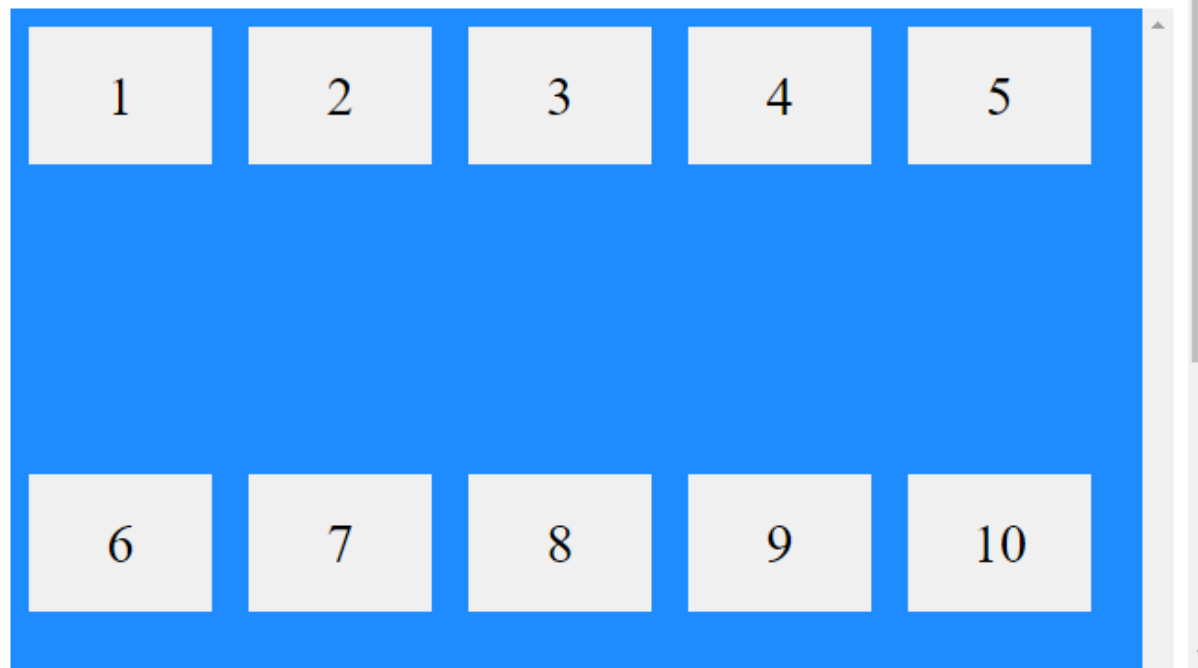
```
<div>12</div>
```

```
</div>

</body>
</html>
```

The align-content Property

The "align-content: space-between;" displays the flex lines with equal space between them:



Example for `space-around`

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  height: 600px;
  flex-wrap: wrap;
  align-content: space-around;
  overflow: scroll;
  background-color: DodgerBlue;
}
```

```
.flex-container > div {  
  background-color: #f1f1f1;  
  width: 100px;  
  margin: 10px;  
  text-align: center;  
  line-height: 75px;  
  font-size: 30px;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>The align-content Property</h1>
```

<p>The "align-content: space-around;" displays the flex lines with space before, between, and after them:</p>

```
<div class="flex-container">
```

```
<div>1</div>
```

```
<div>2</div>
```

```
<div>3</div>
```

```
<div>4</div>
```

```
<div>5</div>
```

```
<div>6</div>
```

```
<div>7</div>
```

```
<div>8</div>
```

```
<div>9</div>
```

```
<div>10</div>
```

```
<div>11</div>
```

```
<div>12</div>
```

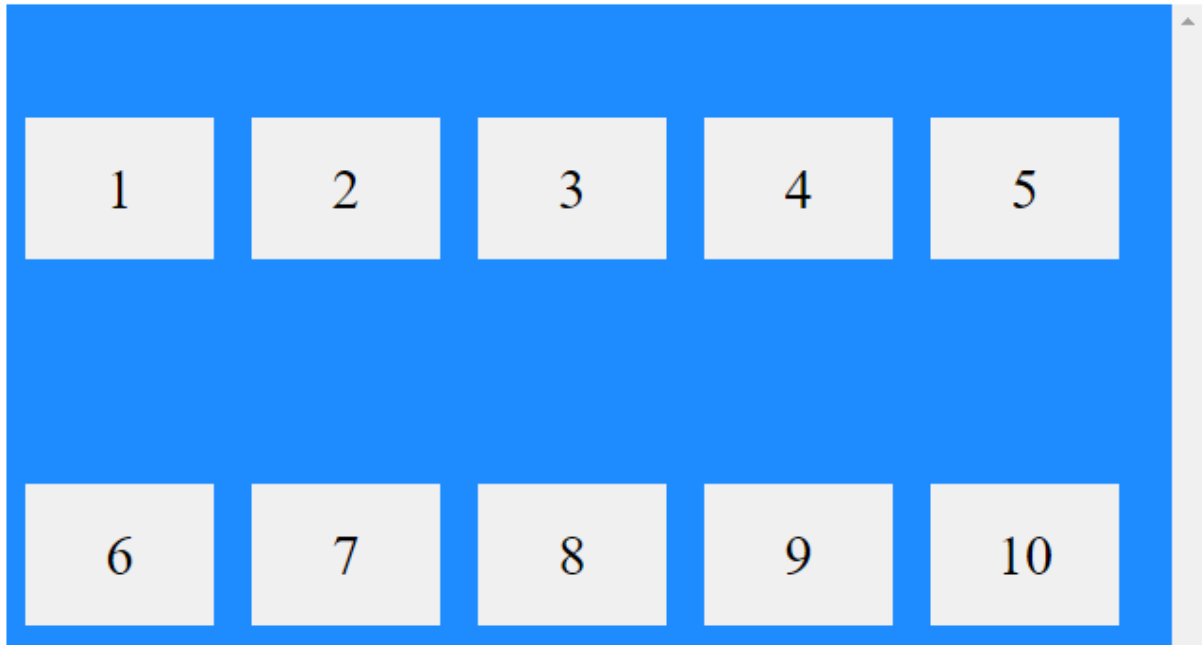
```
</div>
```

```
</body>
```

```
</html>
```

The align-content Property

The "align-content: space-around;" displays the flex lines with space before, between, and after them:



Example for `stretch`

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  height: 600px;
  flex-wrap: wrap;
  align-content: stretch;
  overflow: scroll;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
```



```
text-align: center;
line-height: 75px;
font-size: 30px;
}
</style>
</head>
<body>

<h1>The align-content Property</h1>

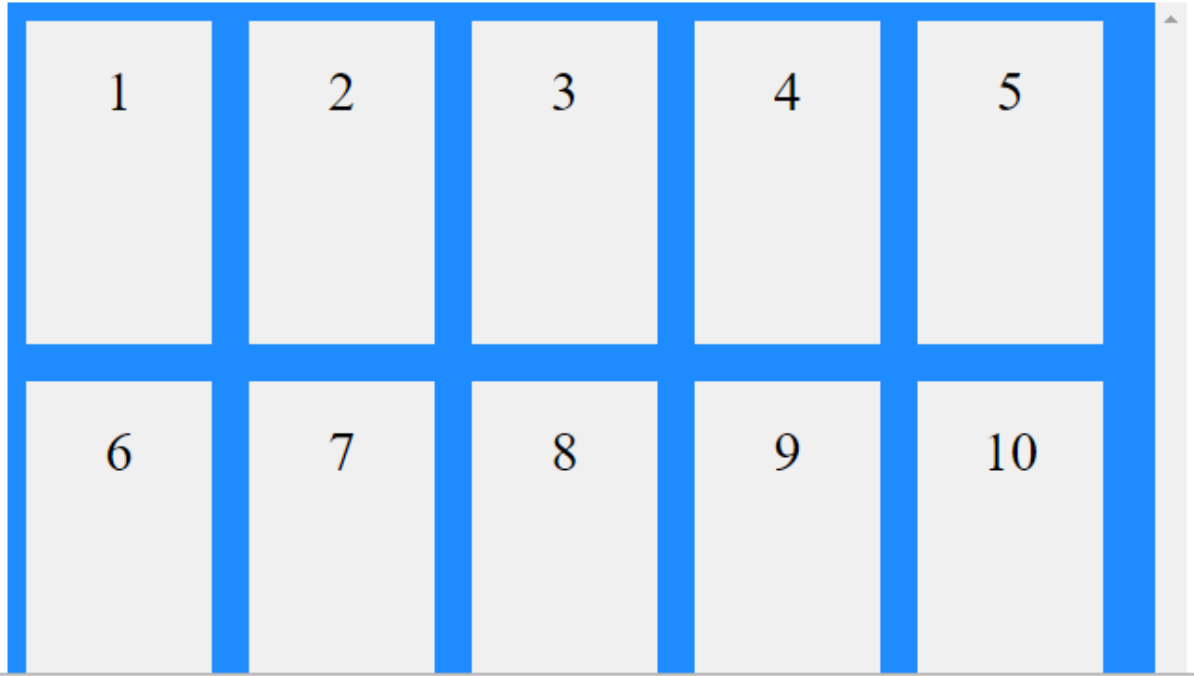
<p>The "align-content: stretch;" stretches the flex lines to take up the remaining space (this is default):</p>

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
  <div>9</div>
  <div>10</div>
  <div>11</div>
  <div>12</div>
</div>

</body>
</html>
```

The align-content Property

The "align-content: stretch;" stretches the flex lines to take up the remaining space (this is default):



Example for **center**

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  height: 600px;
  flex-wrap: wrap;
  align-content: center;
  overflow: scroll;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
```

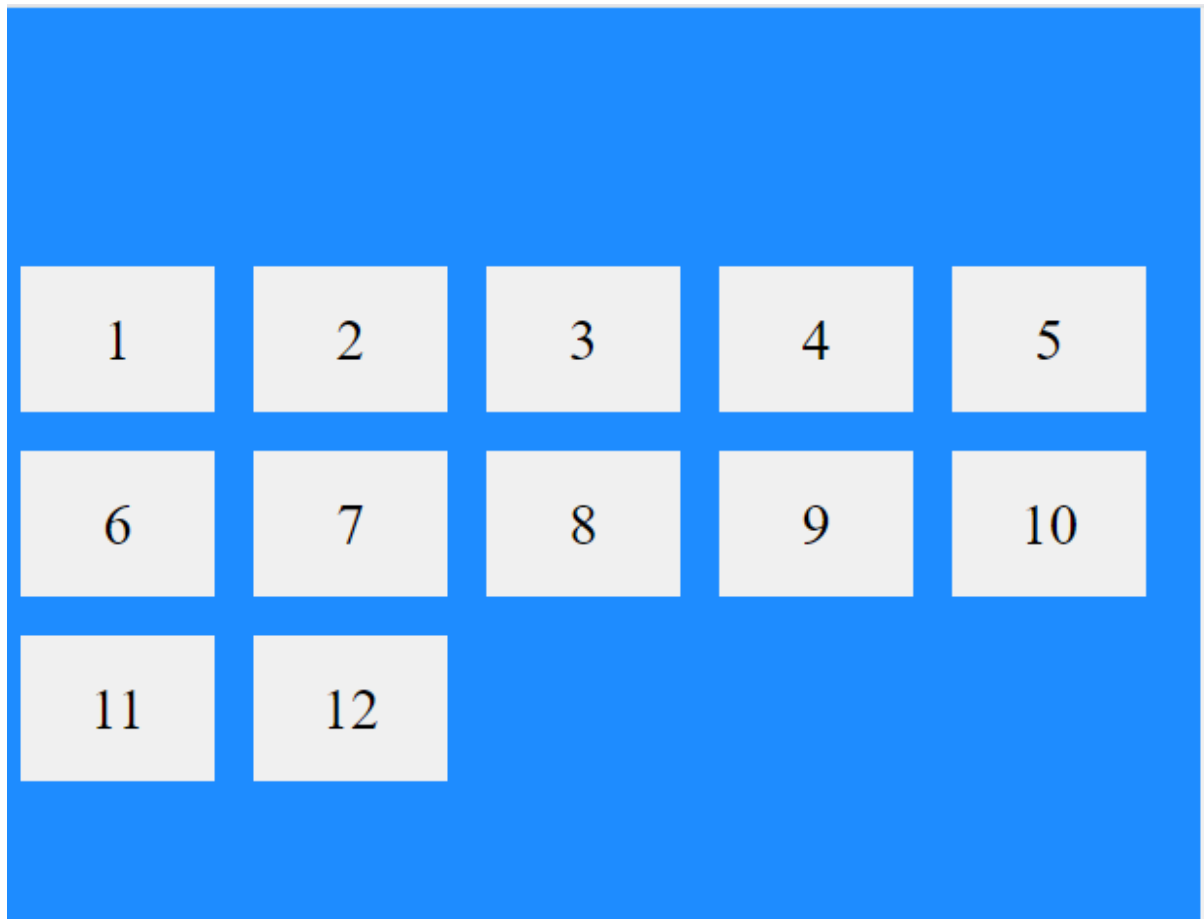
```
text-align: center;
line-height: 75px;
font-size: 30px;
}
</style>
</head>
<body>

<h1>The align-content Property</h1>

<p>The "align-content: center;" displays the flex lines in the middle of the
container:</p>

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
  <div>9</div>
  <div>10</div>
  <div>11</div>
  <div>12</div>
</div>

</body>
</html>
```



Example for **flex-start**

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  height: 600px;
  flex-wrap: wrap;
  align-content: flex-start;
  overflow: scroll;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
```

```
margin: 10px;
text-align: center;
line-height: 75px;
font-size: 30px;
}
</style>
</head>
<body>

<h1>The align-content Property</h1>

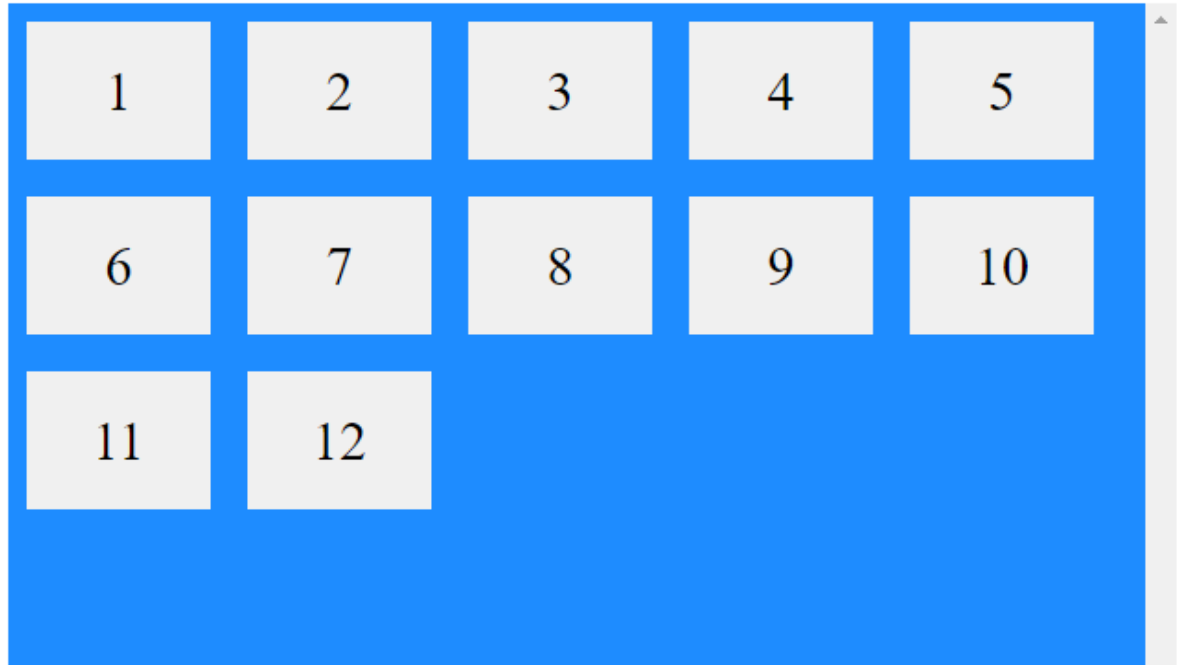
<p>The "align-content: flex-start;" displays the flex lines at the start of the
container:</p>

<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
  <div>9</div>
  <div>10</div>
  <div>11</div>
  <div>12</div>
</div>

</body>
</html>
```

The align-content Property

The "align-content: flex-start;" displays the flex lines at the start of the container:



Example for `flex-end`

```
<!DOCTYPE html>
<html>
<head>
<style>
.flex-container {
  display: flex;
  height: 600px;
  flex-wrap: wrap;
  align-content: flex-end;
  overflow: scroll;
  background-color: DodgerBlue;
}

.flex-container > div {
  background-color: #f1f1f1;
  width: 100px;
  margin: 10px;
```

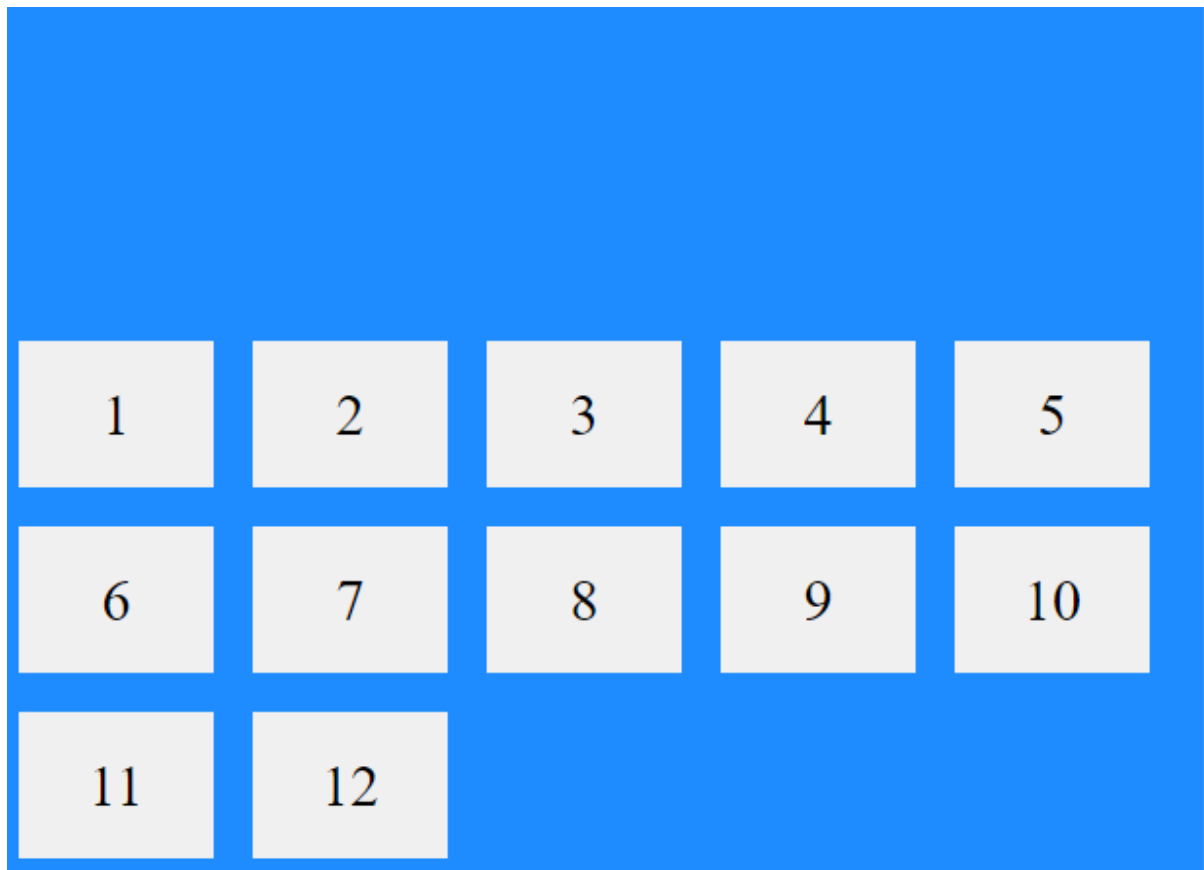
```
text-align: center;
line-height: 75px;
font-size: 30px;
}
</style>
</head>
<body>

<h1>The align-content Property</h1>

<p>The "align-content: flex-end;" displays the flex lines at the end of the container:</p>

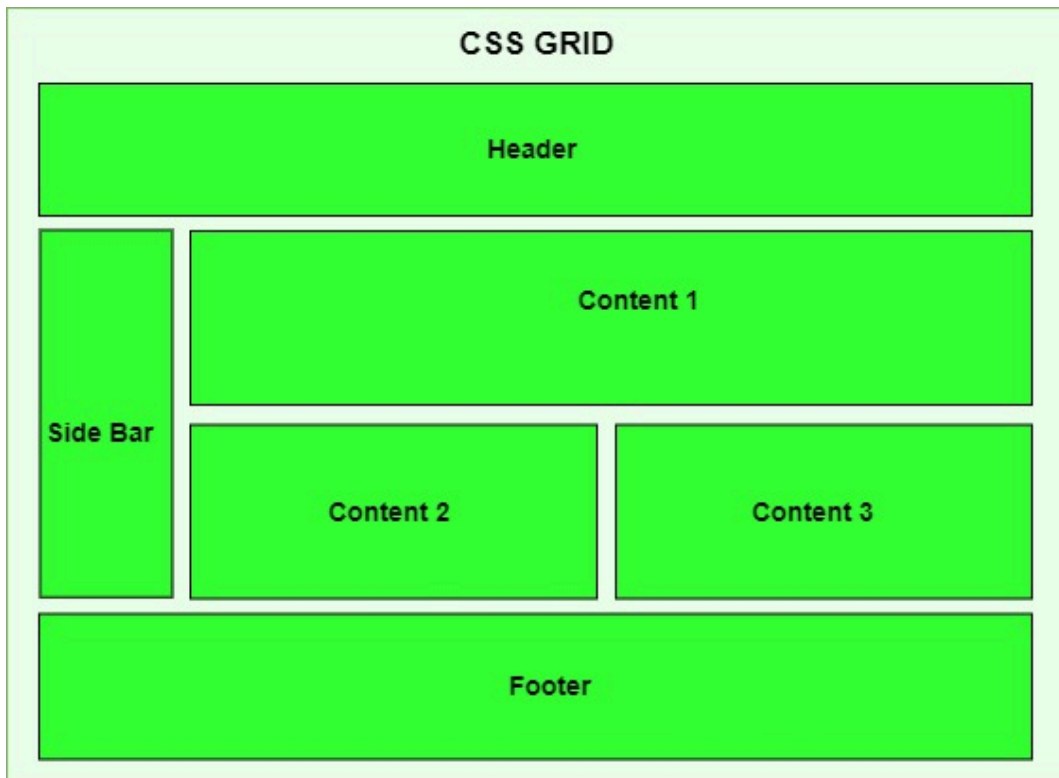
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
  <div>9</div>
  <div>10</div>
  <div>11</div>
  <div>12</div>
</div>

</body>
</html>
```



Grid

- In CSS, a **grid** is a powerful layout system that allows you to create complex and responsive web layouts with minimal code.
- It provides a two-dimensional layout mechanism, meaning you can control both rows and columns of a layout simultaneously.
- It's designed for building layouts with more flexibility and precision compared to older techniques like floats, tables, or inline-block layouts.



Display Property:

An HTML element becomes a grid container when its `display` property is set to `grid` or `inline-grid`.

Different between Grid and Inline-grid:

- `grid`: Block-level grid container that takes up the entire width of its parent and pushes elements above and below it.
- `inline-grid`: Inline-level grid container that only takes up the space its content requires and remains within the inline flow of other elements.

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
}
```


display: grid

Use display: grid; to make a block-level grid container:

1	2	3
4	5	6
7	8	9

Example:

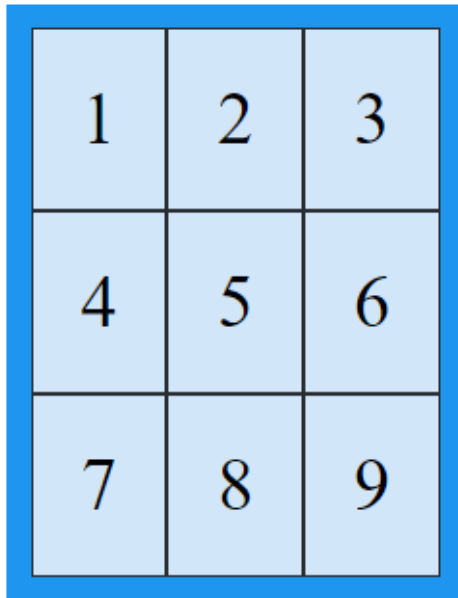
```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: inline-grid;
  grid-template-columns: auto auto auto;
  background-color: #2196F3;
  padding: 10px;
}

.grid-item {
  background-color: rgba(255, 255, 255, 0.8);
  border: 1px solid rgba(0, 0, 0, 0.8);
  padding: 20px;
  font-size: 30px;
  text-align: center;
```

```
}  
</style>  
</head>  
<body>  
  
<h1>display: grid</h1>  
  
<p>Use display: grid; to make a block-level grid container:</p>  
  
<div class="grid-container">  
  <div class="grid-item">1</div>  
  <div class="grid-item">2</div>  
  <div class="grid-item">3</div>  
  <div class="grid-item">4</div>  
  <div class="grid-item">5</div>  
  <div class="grid-item">6</div>  
  <div class="grid-item">7</div>  
  <div class="grid-item">8</div>  
  <div class="grid-item">9</div>  
</div>  
  
</body>  
</html>
```

display: grid

Use `display: grid;` to make a block-level grid container:



grid-template-columns:

The `grid-template-columns` property in CSS defines the number and size of columns in a CSS Grid layout. It allows you to specify how the space within the grid container should be divided into columns, either by using fixed values, relative units, or flexible sizing.

Example:

```
.grid-container {  
  display: grid;
```

```
grid-template-columns: 100px 200px; /* First column is 100px, second is 200px */
}
```

Key Concepts:

1. Fixed Size Columns:

You can define columns using fixed values (e.g., pixels or percentages).

```
.grid-container {
  grid-template-columns: 100px 200px; /* First column is 100px, second is 200px */
}
```

2. Flexible Columns with `fr` Unit:

The `fr` unit stands for "fractional unit" and distributes the available space based on a proportion.

```
.grid-container {
  grid-template-columns: 1fr 2fr; /* First column takes 1 part, second takes 2 parts */
}
```

In this example, the second column will be twice as wide as the first.

3. Repeat Function:

The `repeat()` function is used to create multiple columns with the same size.

```
.grid-container {
  grid-template-columns: repeat( 3, 1fr); /* Creates 3 equal-width columns */
}
```

4. Auto Column Size:

The `auto` keyword lets columns size themselves based on their content.

```
css
Copy code
.grid-container {
  grid-template-columns: auto auto;
  /* Two columns, both sized automatically based on content */
}
```

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: 80px 200px auto 30px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The grid-template-columns Property</h1>

<p>Use the <em>grid-template-columns</em> property to specify the size of each column.</p>

<div class="grid-container">
```

```
<div>1</div>
<div>2</div>
<div>3</div>
<div>4</div>
<div>5</div>
<div>6</div>
<div>7</div>
<div>8</div>
</div>

</body>
</html>
```

Output:

The grid-template-columns Property

Use the *grid-template-columns* property to specify the size of each column.

1	2	3	4
5	6	7	8

grid-template-rows Property

The `grid-template-rows` property in CSS defines the number and height of rows in a

grid layout. It allows you to set the size of each row explicitly, just like how `grid-template-columns` works for columns.

Example:

```
css
Copy code
.grid-container {
  display: grid;
  grid-template-rows: 100px 200px;
  /* First row is 100px, second row is 200px */
}
```

Key Concepts:

1. Fixed Row Heights:

You can set rows to have specific heights using fixed values like pixels.

```
css
Copy code
.grid-container {
  grid-template-rows: 100px 200px;
  /* First row is 100px, second row is 200px */
}
```

2. Flexible Rows with `fr` Unit:

You can use the `fr` unit to allocate available space to rows based on proportions.

```
css
Copy code
.grid-container {
  grid-template-rows: 1fr 2fr;
  /* First row takes 1 part, second row takes 2 parts of available space
  */
}
```

```
}
```

- In this example, the first row will take up 1/3 of the remaining space, and the second row will take up 2/3.

3. Auto-Sizing Rows:

The `auto` keyword makes the row's height adjust based on its content.

```
css
Copy code
.grid-container {
  grid-template-rows: auto auto;
  /* Both rows adjust automatically based on their content */
}
```

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
  grid-template-rows: 80px 200px;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}
</style>
```

```
</head>
<body>

<h1>The grid-template-rows Property</h1>

<div class="grid-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
</div>

<p>Use the <em>grid-template-rows</em> property to specify the size (height) of each row.</p>

</body>
</html>
```

Output:

The grid-template-rows Property

1	2	3
4	5	6

Use the *grid-template-rows* property to specify the size (height) of each row.

Grid Gap:

In CSS Grid, **grid gaps** define the space between rows and columns in the grid layout. This allows you to add spacing between grid items without needing to manually add margins or padding to individual grid items.

There are two main properties for defining gaps:

1. **gap** (or the shorthand for both row and column gaps)
2. **row-gap** (for controlling the gap between rows)
3. **column-gap** (for controlling the gap between columns)

Example for column gap:

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  column-gap: 50px;
```

```
grid-template-columns: auto auto auto;  
background-color: #2196F3;  
padding: 10px;  
}
```

```
.grid-item {  
background-color: rgba(255, 255, 255, 0.8);  
border: 1px solid rgba(0, 0, 0, 0.8);  
padding: 20px;  
font-size: 30px;  
text-align: center;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>The column-gap Property</h1>
```

<p>Use the column-gap property to adjust the space between the columns:</p>

```
<div class="grid-container">  
  <div class="grid-item">1</div>  
  <div class="grid-item">2</div>  
  <div class="grid-item">3</div>  
  <div class="grid-item">4</div>  
  <div class="grid-item">5</div>  
  <div class="grid-item">6</div>  
  <div class="grid-item">7</div>  
  <div class="grid-item">8</div>  
  <div class="grid-item">9</div>  
</div>
```

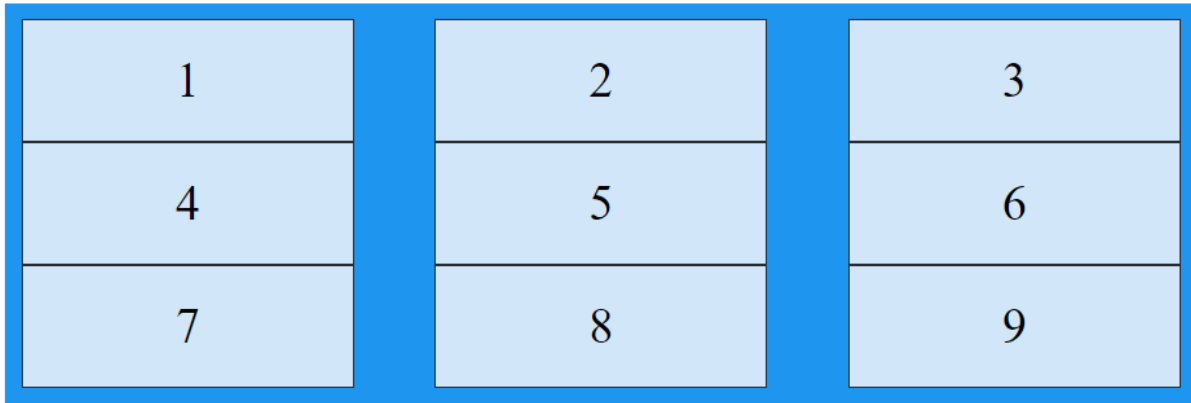
```
</body>
```

```
</html>
```

Output:

The column-gap Property

Use the *column-gap* property to adjust the space between the columns:



Example for row gap

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  row-gap: 50px;
  grid-template-columns: auto auto auto;
  background-color: #2196F3;
  padding: 10px;
}

.grid-item {
  background-color: rgba(255, 255, 255, 0.8);
  border: 1px solid rgba(0, 0, 0, 0.8);
  padding: 20px;
```

```
font-size: 30px;
text-align: center;
}
</style>
</head>
<body>

<h1>The row-gap Property</h1>

<p>Use the <em>row-gap</em> property to adjust the space between the
rows:</p>

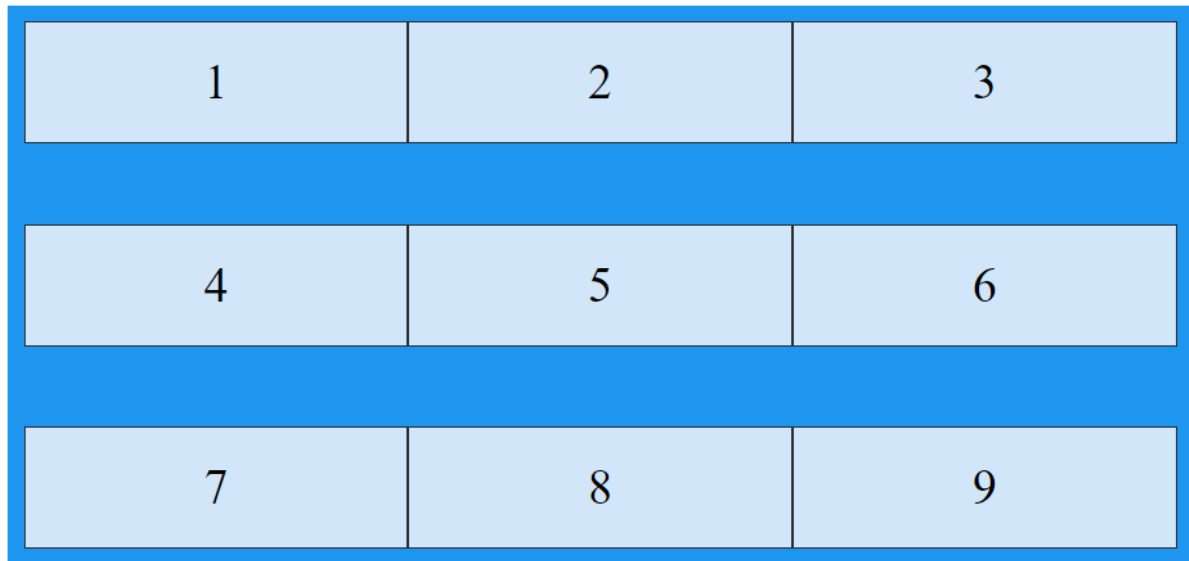
<div class="grid-container">
  <div class="grid-item">1</div>
  <div class="grid-item">2</div>
  <div class="grid-item">3</div>
  <div class="grid-item">4</div>
  <div class="grid-item">5</div>
  <div class="grid-item">6</div>
  <div class="grid-item">7</div>
  <div class="grid-item">8</div>
  <div class="grid-item">9</div>
</div>

</body>
</html>
```

Output:

The row-gap Property

Use the *row-gap* property to adjust the space between the rows:



Example for Gap:

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  gap: 50px 100px;
  grid-template-columns: auto auto auto;
  background-color: #2196F3;
  padding: 10px;
}

.grid-item {
  background-color: rgba(255, 255, 255, 0.8);
  border: 1px solid rgba(0, 0, 0, 0.8);
  padding: 20px;
```



```
font-size: 30px;
text-align: center;
}
</style>
</head>
<body>

<h1>The gap Property</h1>

<p>Use the <em>gap</em> shorthand property to adjust the space between the columns and rows.</p>

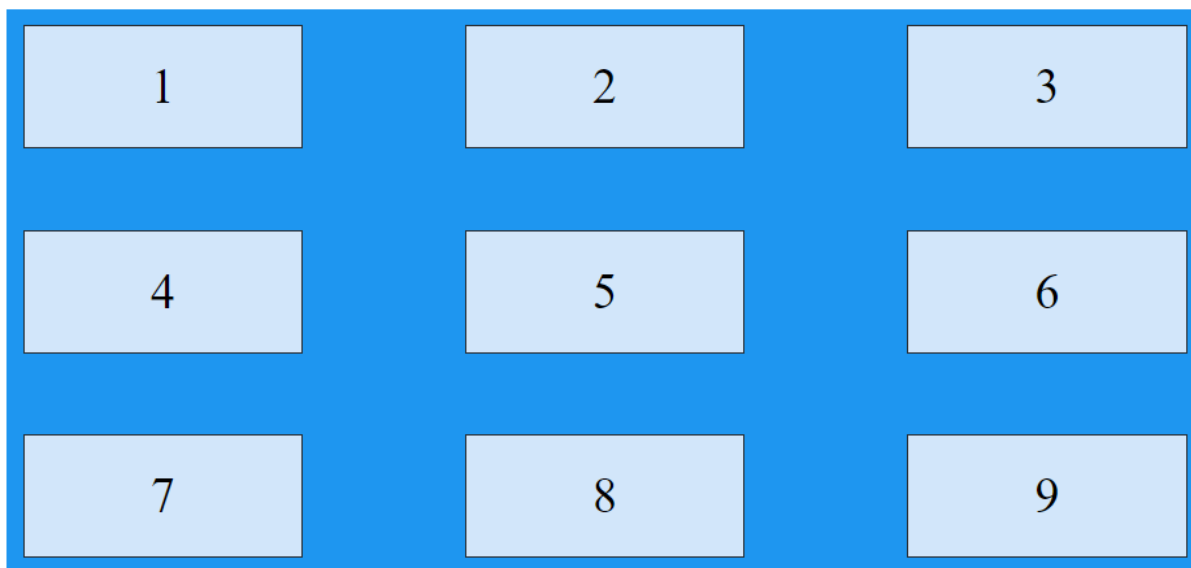
<div class="grid-container">
  <div class="grid-item">1</div>
  <div class="grid-item">2</div>
  <div class="grid-item">3</div>
  <div class="grid-item">4</div>
  <div class="grid-item">5</div>
  <div class="grid-item">6</div>
  <div class="grid-item">7</div>
  <div class="grid-item">8</div>
  <div class="grid-item">9</div>
</div>

</body>
</html>
```

Output:

The gap Property

Use the *gap* shorthand property to adjust the space between the columns and rows.



justify-content Property:

same as flex

align-content Property

same as flex

grid-column Property:

- The `grid-column` property in CSS is used to control how many columns a grid item spans and its placement within a grid container.
- It allows you to specify the starting and ending positions of a grid item along the **horizontal axis** (columns) of the grid layout.

Syntax:

```
grid-column: start / end;
```

- **start**: The starting column line where the grid item begins.

- **end**: The ending column line where the grid item ends (not inclusive of this line).

Values:

1. **start / end**:
 - Numeric values that refer to the grid lines.
 - Example: `grid-column: 1 / 3;` (starts at column line 1 and spans to column line 3).
2. **span**:
 - Allows the item to span a certain number of columns.
 - Example: `grid-column: span 2;` (spans 2 columns from its starting position).
3. **auto**:
 - The browser will automatically place the grid item based on the available space.
 - Example: `grid-column: auto;` .

Example 1: Basic Column Span

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto auto auto auto;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
```

```
font-size: 30px;
}

.item1 {
  grid-column: 1 / 5;
}

</style>
</head>
<body>
```

```
<h1>The grid-column Property</h1>
```

```
<p>Use the <em>grid-column</em> property to specify where to place an
item.</p>
```

```
<p>Item1 will start on column 1 and end before column 5.</p>
```

```
<div class="grid-container">
  <div class="item1">1</div>
  <div class="item2">2</div>
  <div class="item3">3</div>
  <div class="item4">4</div>
  <div class="item5">5</div>
  <div class="item6">6</div>
  <div class="item7">7</div>
  <div class="item8">8</div>
  <div class="item9">9</div>
  <div class="item10">10</div>
  <div class="item11">11</div>
  <div class="item12">12</div>
  <div class="item13">13</div>
  <div class="item14">14</div>
  <div class="item15">15</div>
</div>

</body>
</html>
```

The grid-column Property

Use the *grid-column* property to specify where to place an item.

Item1 will start on column 1 and end before column 5:

1				2	3
4	5	6	7	8	9
10	11	12	13	14	15

Example 2: Span a Specific Number of Columns

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto auto auto auto;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
```

```
font-size: 30px;
}

.item1 {
  grid-column: 1 / span 3;
}

</style>
</head>
<body>
```

<h1>The grid-column Property</h1>

<p>Use the grid-column property to specify where to place an item.</p>

<p>Item1 will start on column-line 1 and span 3 columns:</p>

```
<div class="grid-container">
  <div class="item1">1</div>
  <div class="item2">2</div>
  <div class="item3">3</div>
  <div class="item4">4</div>
  <div class="item5">5</div>
  <div class="item6">6</div>
  <div class="item7">7</div>
  <div class="item8">8</div>
  <div class="item9">9</div>
  <div class="item10">10</div>
  <div class="item11">11</div>
  <div class="item12">12</div>
  <div class="item13">13</div>
  <div class="item14">14</div>
  <div class="item15">15</div>
  <div class="item16">16</div>
</div>

</body>
</html>
```

The grid-column Property

Use the *grid-column* property to specify where to place an item.

Item1 will start on column-line 1 and span 3 columns:

1			2	3	4
5	6	7	8	9	10
11	12	13	14	15	16

Example 3: Automatic Placement

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto auto auto auto;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
```

```
background-color: rgba(255, 255, 255, 0.8);
text-align: center;
padding: 20px 0;
font-size: 30px;
}
```

```
.item1 {
  grid-column: auto
</style>
</head>
<body>
```

```
<h1>The grid-column Property</h1>
```

<p>Use the `grid-column` property to specify where to place an item.</p>

<p>Item1 will start on column-line 1 and span 3 `columns`:</p>

```
<div class="grid-container">
```

```
  <div class="item1">The grid-column property in CSS is used to control how many columns a grid item spans and its placement within a grid container. It allows you to specify the starting and ending positions of a grid item along the horizontal axis (columns) of the grid layout.</div>
```

```
  <div class="item2">2</div>
```

```
  <div class="item3">3</div>
```

```
  <div class="item4">4</div>
```

```
  <div class="item5">5</div>
```

```
  <div class="item6">6</div>
```

```
  <div class="item7">7</div>
```

```
  <div class="item8">8</div>
```

```
  <div class="item9">9</div>
```

```
  <div class="item10">10</div>
```

```
  <div class="item11">11</div>
```

```
  <div class="item12">12</div>
```

```
  <div class="item13">13</div>
```

```
  <div class="item14">14</div>
```

```
  <div class="item15">15</div>
```

```
  <div class="item16">16</div>
```



```
</div>

</body>
</html>
```

Output:

The grid-column property in CSS is used to control how many columns a grid item spans and its placement within a grid container. It allows you to specify the starting and ending positions of a grid item along the horizontal axis (columns) of the grid layout.	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16		

Note: The `grid-column` property is a shorthand property for the `grid-column-start` and the `grid-column-end` properties.

The `grid-row` property in CSS is used to control how many rows a grid item spans and where it is positioned along the **vertical axis** (rows) within a grid layout. Similar to `grid-column`, it allows you to define the starting and ending positions of a grid item, but vertically.

Syntax:

grid-row: start / end;

- **start** : The starting row line where the grid item begins.
- **end** : The ending row line where the grid item ends (not inclusive of this line).

Values: **start** / **end** :

1. **start** / **end** :

- Numeric values that correspond to grid row lines.
- Example: **grid-row: 1 / 3;** (starts at row line 1 and spans to row line 3).

2. **span** :

- Specifies how many rows the item should span.
- Example: **grid-row: span 2;** (spans 2 rows from its starting point).

3. **auto** :

- Allows the browser to automatically place the grid item based on the available space.
- Example: **grid-row: auto;** .

Example 1: Basic Row Span

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto auto auto auto;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}
```

```
.grid-container > div {  
  background-color: rgba(255, 255, 255, 0.8);  
  text-align: center;  
  padding: 20px 0;  
  font-size: 30px;  
}
```

```
.item1 {  
  grid-row: 1 / 4;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>The grid-row Property</h1>
```

<p>Use the grid-row property to specify where to place an item.</p>

<p>Item1 will start on row-line 1 and end on row-line 4.</p>

```
<div class="grid-container">  
  <div class="item1">1</div>  
  <div class="item2">2</div>  
  <div class="item3">3</div>  
  <div class="item4">4</div>  
  <div class="item5">5</div>  
  <div class="item6">6</div>  
  <div class="item7">7</div>  
  <div class="item8">8</div>  
  <div class="item9">9</div>  
  <div class="item10">10</div>  
  <div class="item11">11</div>  
  <div class="item12">12</div>  
  <div class="item13">13</div>  
  <div class="item14">14</div>  
  <div class="item15">15</div>  
  <div class="item16">16</div>
```

```
</div>

</body>
</html>
```

The grid-row Property

Use the *grid-row* property to specify where to place an item.

Item1 will start on row-line 1 and end on row-line 4:

1	2	3	4	5	6
	7	8	9	10	11
	12	13	14	15	16

Example 2: Span a Specific Number of Rows

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto auto auto auto;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
```

```

}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}

```

```

.item1 {
  grid-row: 1 / span 2;
}

```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>The grid-row Property</h1>
```

<p>Use the grid-row property to specify where to place an item.</p>

<p>Item1 will start on row 1 and span 2 rows:</p>

```

<div class="grid-container">
  <div class="item1">1</div>
  <div class="item2">2</div>
  <div class="item3">3</div>
  <div class="item4">4</div>
  <div class="item5">5</div>
  <div class="item6">6</div>
  <div class="item7">7</div>
  <div class="item8">8</div>
  <div class="item9">9</div>
  <div class="item10">10</div>
  <div class="item11">11</div>
  <div class="item12">12</div>
  <div class="item13">13</div>
  <div class="item14">14</div>
  <div class="item15">15</div>

```

```
<div class="item16">16</div>
<div class="item17">17</div>
</div>

</body>
</html>
```

The grid-row Property

Use the *grid-row* property to specify where to place an item.

Item1 will start on row 1 and span 2 rows:

1	2	3	4	5	6
	7	8	9	10	11
12	13	14	15	16	17

Example 3: Automatic Row Placement

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto auto auto auto;
  gap: 10px;
  background-color: #2196F3;
```

```
padding: 10px;
}

.grid-container > div {
background-color: rgba(255, 255, 255, 0.8);
text-align: center;
padding: 20px 0;
font-size: 30px;
}
```

```
.item1 {
grid-row: auto
}
```

```
</style>
</head>
<body>
```

```
<h1>The grid-row Property</h1>
```

<p>Use the grid-row property to specify where to place an item.</p>

<p>Item1 will start on row 1 and span 2 rows:</p>

```
<div class="grid-container">
  <div class="item1">1</div>
  <div class="item2">2</div>
  <div class="item3">3</div>
  <div class="item4">4</div>
  <div class="item5">5</div>
  <div class="item6">6</div>
  <div class="item7">7</div>
  <div class="item8">8</div>
  <div class="item9">9</div>
  <div class="item10">10</div>
  <div class="item11">11</div>
  <div class="item12">12</div>
  <div class="item13">13</div>
  <div class="item14">14</div>
```

```

<div class="item15">15</div>
<div class="item16">16</div>
<div class="item17">17</div>
</div>

</body>
</html>

```

The grid-row Property

Use the *grid-row* property to specify where to place an item.

Item1 will start on row 1 and span 2 rows:

In this example, the grid item will span rows 2 and 3, starting at row line 2 and ending at row line 4, in a grid with four rows.	2	3	4	5	6
	7	8	9	10	11
13	14	15	16	17	

grid-area Property:

- The `grid-area` property in CSS is a shorthand property that allows you to define where a grid item should be placed within a grid container.
- It combines the functionality of the `grid-row-start`, `grid-row-end`, `grid-column-start`, and `grid-column-end` properties, making it a more concise way to define both the row and column positioning for a grid item.

Syntax:

grid-area: row-start / column-start / row-end / column-end;

- **row-start**: The line where the grid item starts vertically (row).
- **column-start**: The line where the grid item starts horizontally (column).
- **row-end**: The line where the grid item ends vertically (row).
- **column-end**: The line where the grid item ends horizontally (column).

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto auto auto auto;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}

.item8 {
  grid-area: 1 / 2 / 5 / 6;
}
</style>
</head>
<body>
```

<h1>The grid-area Property</h1>

<p>You can use the grid-area property to specify where to place an item.</p>

<p>The syntax is:</p>

<p>grid-row-start / grid-column-start / grid-row-end / grid-column-end.</p>

<p>Item8 will start on row-line 1 and column-line 2, and end on row-line 5 column-line 6:</p>

```
<div class="grid-container">
  <div class="item1">1</div>
  <div class="item2">2</div>
  <div class="item3">3</div>
  <div class="item4">4</div>
  <div class="item5">5</div>
  <div class="item6">6</div>
  <div class="item7">7</div>
  <div class="item8">8</div>
  <div class="item9">9</div>
  <div class="item10">10</div>
  <div class="item11">11</div>
  <div class="item12">12</div>
  <div class="item13">13</div>
  <div class="item14">14</div>
  <div class="item15">15</div>
</div>
```

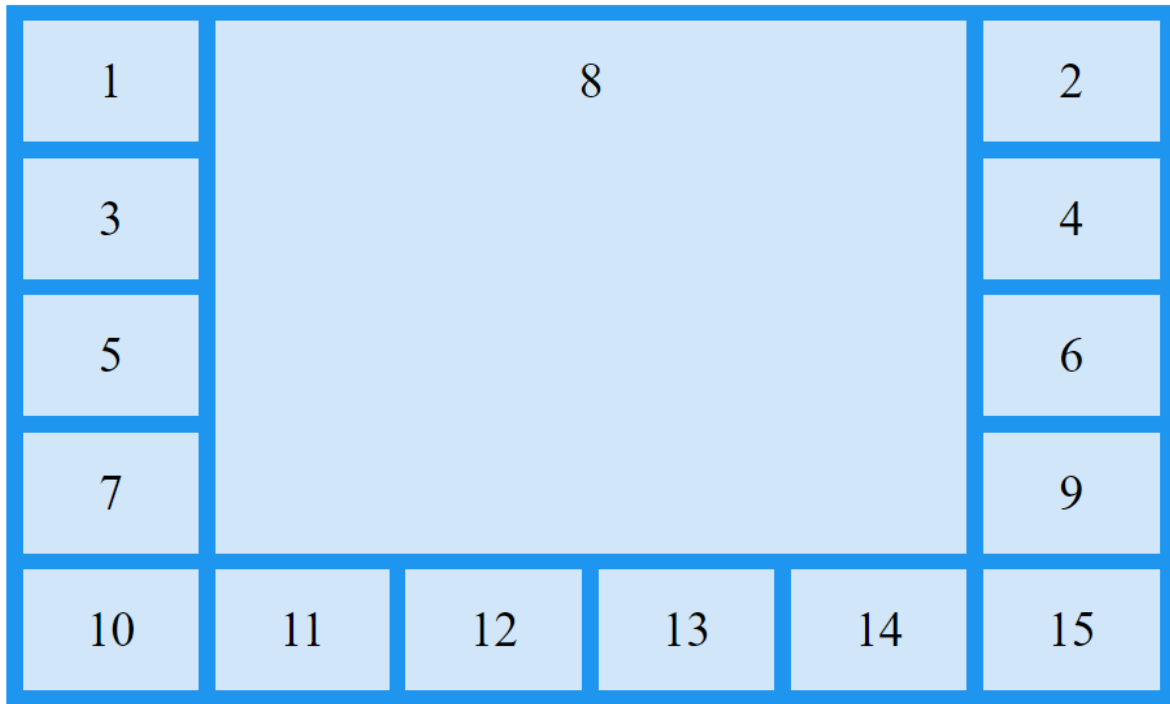
</body>

</html>

The syntax is:

grid-row-start / grid-column-start / grid-row-end / grid-column-end.

Item8 will start on row-line 1 and column-line 2, and end on row-line 5 column-line 6:



Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto auto auto auto;
  gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}
```

```

}

.item8 {
  grid-area: 2 / 1 / span 2 / span 3;
}
</style>
</head>
<body>

<h1>The grid-area Property</h1>

<p>You can use the grid-area property to specify where to place an item.</p>

<p>The syntax is grid-row-start / grid-column-start / grid-row-end / grid-column-end.</p>

<p>Item8 will start on row-line 2 and column-line 1, and span 2 rows and 3 columns:</p>

<div class="grid-container">
  <div class="item1">1</div>
  <div class="item2">2</div>
  <div class="item3">3</div>
  <div class="item4">4</div>
  <div class="item5">5</div>
  <div class="item6">6</div>
  <div class="item7">7</div>
  <div class="item8">8</div>
  <div class="item9">9</div>
  <div class="item10">10</div>
  <div class="item11">11</div>
  <div class="item12">12</div>
  <div class="item13">13</div>
</div>

</body>
</html>

```

The grid-area Property

You can use the *grid-area* property to specify where to place an item.

The syntax is `grid-row-start / grid-column-start / grid-row-end / grid-column-end`.

Item8 will start on row-line 2 and column-line 1, and span 2 rows and 3 columns:

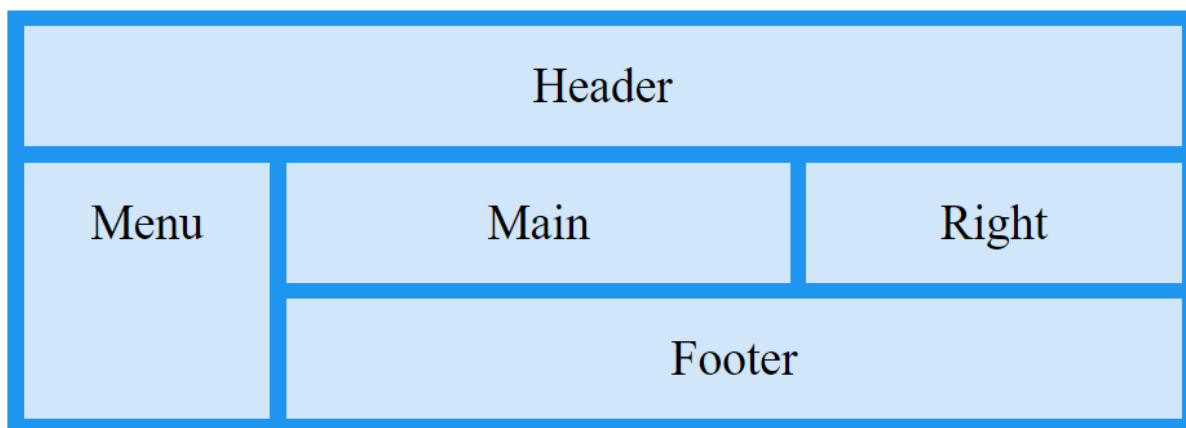
1	2	3	4	5	6
8			7	9	10
			11	12	13

The grid-area Property

You can use the *grid-area* property to name grid items.

You can refer to the name when you set up the grid layout, by using the *grid-template-areas* property on the grid container.

This grid layout contains six columns and three rows:



Transition

CSS transitions allows you to change property values smoothly, over a given duration.

- `transition`
- `transition-delay`
- `transition-duration`
- `transition-property`
- `transition-timing-function`

How to Use CSS Transitions?

To create a transition effect, you must specify two things:

- the CSS property you want to add an effect to
- the duration of the effect

Note: If the duration part is not specified, the transition will have no effect, because the default value is 0.

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 100px;
  height: 100px;
  background: red;
  transition: width 2s;
}

div:hover {
  width: 300px;
}
</style>
</head>
<body>

<h1>The transition Property</h1>

<p>Hover over the div element below, to see the transition effect:</p>
<div></div>

</body>
</html>
```

After Hover

Another Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 100px;
```

```
height: 100px;
background: red;
transition: width 2s, height 4s;
}

div:hover {
width: 300px;
height: 300px;
}
</style>
</head>
<body>

<h1>The transition Property</h1>

<p>Hover over the div element below, to see the transition effect:</p>

<div></div>

</body>
</html>
```

Specify the Speed Curve of the Transition

The `transition-timing-function` property specifies the speed curve of the transition effect.

The transition-timing-function property can have the following values:

- `ease` - specifies a transition effect with a slow start, then fast, then end slowly (this is default)
- `linear` - specifies a transition effect with the same speed from start to end
- `ease-in` - specifies a transition effect with a slow start
- `ease-out` - specifies a transition effect with a slow end
- `ease-in-out` - specifies a transition effect with a slow start and end

Example:


```

<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 100px;
  height: 100px;
  background: red;
  transition: width 2s;
}

#div1 {transition-timing-function: linear;}
#div2 {transition-timing-function: ease;}
#div3 {transition-timing-function: ease-in;}
#div4 {transition-timing-function: ease-out;}
#div5 {transition-timing-function: ease-in-out;}

div:hover {
  width: 300px;
}
</style>
</head>
<body>

<h1>The transition-timing-function Property</h1>

<p>Hover over the div elements below, to see the different speed curves:
</p>

<div id="div1">linear</div><br>
<div id="div2">ease</div><br>
<div id="div3">ease-in</div><br>
<div id="div4">ease-out</div><br>
<div id="div5">ease-in-out</div><br>

</body>
</html>

```

Output :

run the above program and see output

Delay the Transition Effect

The `transition-delay` property specifies a delay (in seconds) for the transition effect.

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 100px;
  height: 100px;
  background: red;
  transition: width 3s;
  transition-delay: 1s;
}

div:hover {
  width: 300px;
}
</style>
</head>
<body>

<h1>The transition-delay Property</h1>

<p>Hover over the div element below, to see the transition effect:</p>

<div></div>

<p><b>Note:</b> The transition effect has a 1 second delay before starting.</p>
```

```
</body>
</html>
```

Transition + Transformation

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 100px;
  height: 100px;
  background: red;
  transition: width 2s, height 2s, transform 5s;
}

div:hover {
  width: 300px;
  height: 300px;
  transform: rotate(180deg);
}
</style>
</head>
<body>

<h1>Transition + Transform</h1>

<p>Hover over the div element below:</p>

<div></div>

</body>
</html>
```

Transition shorthand property:

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 100px;
  height: 100px;
  background: red;
  transition: width 2s linear 1s;
}

div:hover {
  width: 300px;
}
</style>
</head>
<body>

<h1>Using The transition Shorthand Property</h1>

<p>Hover over the div element below, to see the transition effect:</p>

<div></div>

<p><b>Note:</b> The transition effect has a 1 second delay before starting.</p>

</body>
</html>
```